

Base de Datos II

Diseño e Implementación de un Motor de Base de Datos

Indexación en Memoria Secundaria y Parser SQL

Integrantes: Alvarez, Lanche, Melgarejo, Pachacutec, Yangali

Docente: Heider Ysaías Sanchez Enriquez

Introducción al Proyecto

BASE DE DATOS 2

Mini DBMS / Frontend

QUERY EDITOR

SQL Query

User 1 ✕ User 2 ✕ +

```
1 -- 3.1 Prueba de Llave Primaria (Sequential)
2 SELECT * FROM cities WHERE id = 3;
3 SELECT * FROM cities WHERE id BETWEEN 2 AND 5;
4
5 -- 3.2 Prueba de Extendible Hashing (Exacto)
6 SELECT * FROM cities WHERE state_name = "La Massana";
7
8 -- 3.3 Prueba de B+ Tree (Rangos y exacto)
9 SELECT * FROM cities WHERE state_id = 489;
10 SELECT * FROM cities WHERE state_id BETWEEN 489 AND 493;
11
12 -- 3.4 Prueba de R-Tree (Espacial)
13 SELECT * FROM cities WHERE location IN (POINT(-27.31676, -58.57197),
14     RADIUS 0.5);
15 SELECT * FROM cities WHERE location IN (POINT(-27.31676, -58.57197), K
16     2);
17
18 -- 3.5 Prueba de Full Table Scan (Columna sin indice)
19 SELECT * FROM cities WHERE name = "Riachuelo";
```

Tip: Usa multiples pestañas para organizar tus consultas.

Ejecutar todos Ejecutar

CATALOG

Datasets 1 csv

datasets/

cities_1.csv

Sube archivos .csv para trabajar. Subir CSV

ACTIVITY

Results Logs

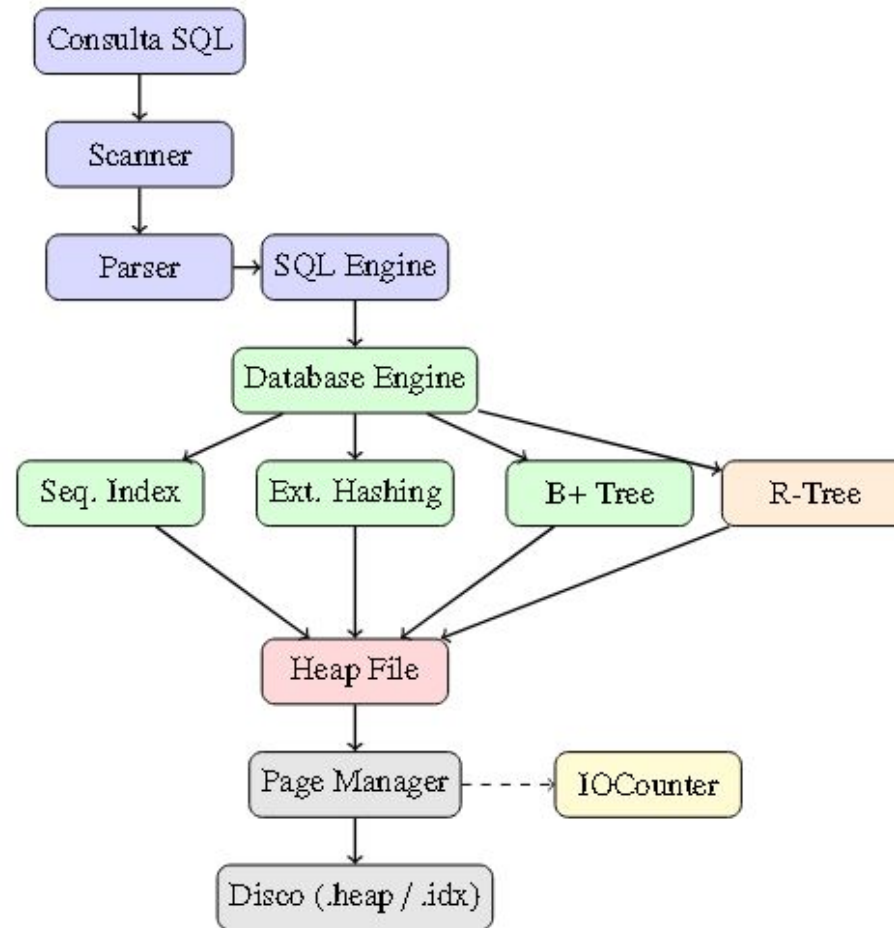
id	name	state_id	state_code	state_name	count
1382	Riachuelo	3638	W	Corrientes	11

Muestra los resultados tabulares de la consulta. Reiniciar

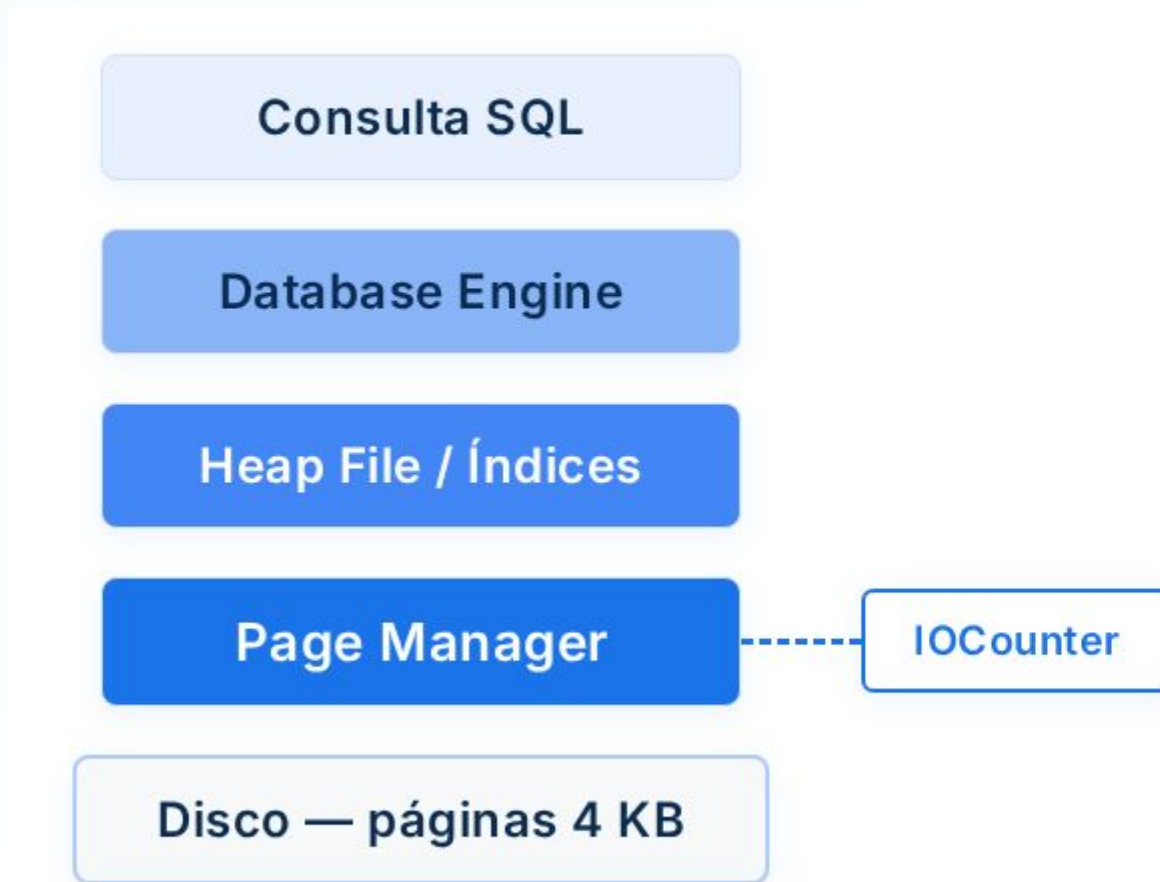
Desarrollo de un **SGBD** a escala reducida centrado en la organización eficiente de archivos en memoria secundaria.

- ✓ Gestión estricta de páginas de **4 KB**.
- ✓ Implementación de estructuras desde cero.
- ✓ Soporte para subconjunto de lenguaje **SQL**.
- ✓ Simulador de concurrencia e interfaz web.

Arquitectura de integración



La Base: ¿Cómo tocamos el disco?



* Toda E/S pasa por el PageManager — nadie accede al disco directamente

Heap File vs Sequential Index

Heap File

-  Almacena registros completos en formato binario.
-  Sin orden preestablecido — garantiza máxima velocidad de escritura.
-  **Free-list:** reutiliza eficientemente el espacio de slots eliminados.
-  Inserción y búsqueda directa por RID: **1 lectura**.

Sequential Index

-  Lista enlazada ordenada lógicamente por Primary Key (PK).
-  Nodo = (PK + RID + next_pos) = 16 bytes → **255 nodos/página**.
-  **Sparse Index en RAM:** búsqueda binaria antes de tocar el disco.
-  Área auxiliar + Reconstrucción automática tipo merge cuando $k_{aux} \geq \lg(n)$.

Extendible Hashing

Estructura Física del Bucket



Header (12 bytes)

Almacena local_depth, num_records y next_page_id.



Data Body

Almacena registros de tamaño fijo (Key + RID).



Capacidad Dinámica

Calculada según el formato de la columna indexada.

Uso del MD5



Anti-Clustering

MD5 dispersa llaves secuenciales en buckets alejados, garantizando un llenado uniforme.



Bit Masking

Extracción instantánea del bucket destino mediante máscaras de bits de bajo nivel.

Gestión de Profundidad y Límites en RAM

PROFUNDIDAD GLOBAL Y CHAINING

Prevención de RAM Overflow

Cálculo dinámico del **MAX_GLOBAL_DEPTH** basado en la expectativa de carga de la tabla (100k registros).

Permite que el directorio crezca de forma segura hasta **512 punteros (2⁹)**.

Superado este límite, el sistema aplica automáticamente **Chaining**.

PROCESO SPLIT

Algoritmo de Reestructuración

01

Patrón Binario: Identificación del patrón de bits compartido en el bucket saturado.

02

Redistribución: Evaluación del bit extra para dividir registros en b0 y b1.

03

Escritura Disco: Bucket 0 reutiliza páginas; Bucket 1 solicita nuevo ID al Page Manager.

04

Actualización: Ajuste de punteros del directorio que coinciden con el nuevo patrón.

B+ Tree

Estructura de datos de búsqueda jerárquica con balanceo dinámico. La evolución del árbol B

Estructura Paginada



Nodes

Unidad mínima <> Nodo <> Página (4kb)



LeafNode

Header + Data (RID) + next_leaf



Internal Node

Header + Children

Gestión de llaves



ID <> Key <> Columna

Soporte de tamaño variable con padding nulo y codificación UTF-8.



Duplicados

Bisect_right y bisect_left para optimización

Headers



BPlusHeader

Página 0 y actúa como el punto de entrada



BPlusNodeHeader

Si es una hoja, cuántas llaves tiene guardadas y quién es su padre

| Indexación Espacial: R-Tree

Soporte fundamental para datos geográficos y consultas multidimensionales.



Consulta kNN

Búsqueda eficiente de los K vecinos más cercanos en espacios multidimensionales.



Consultas de Radio

Optimización de búsquedas por rango circular para datos geográficos.



Coordenadas

Indexación nativa de Latitud y Longitud para soporte geoespacial.

RtreeFile.dat

P0: FileHeaders

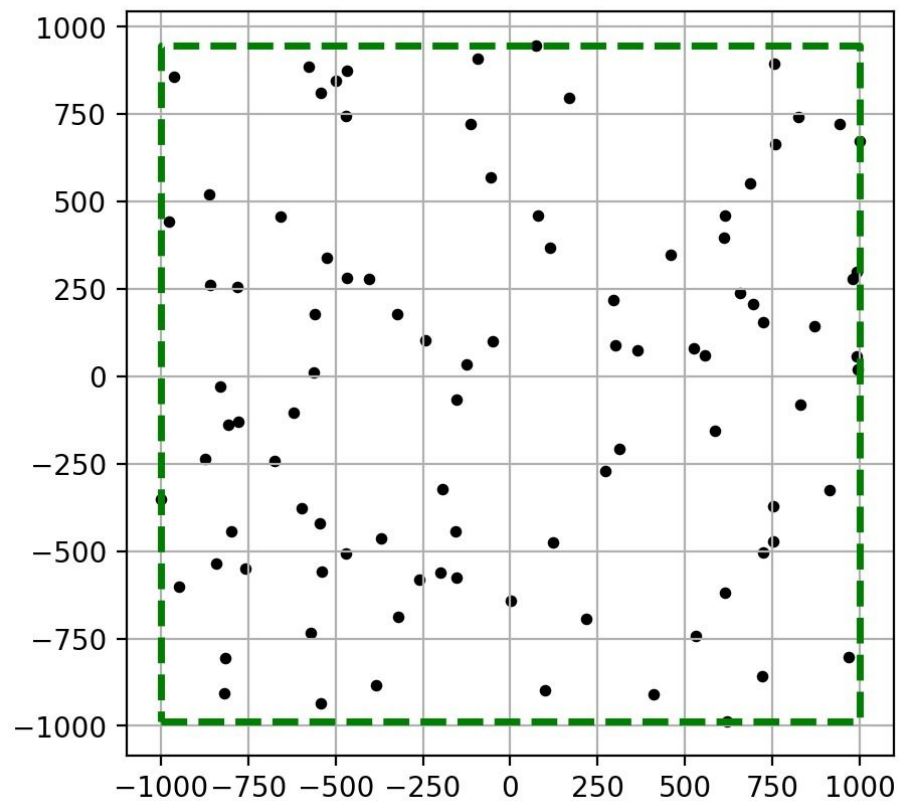
P1: Node

P2: Node

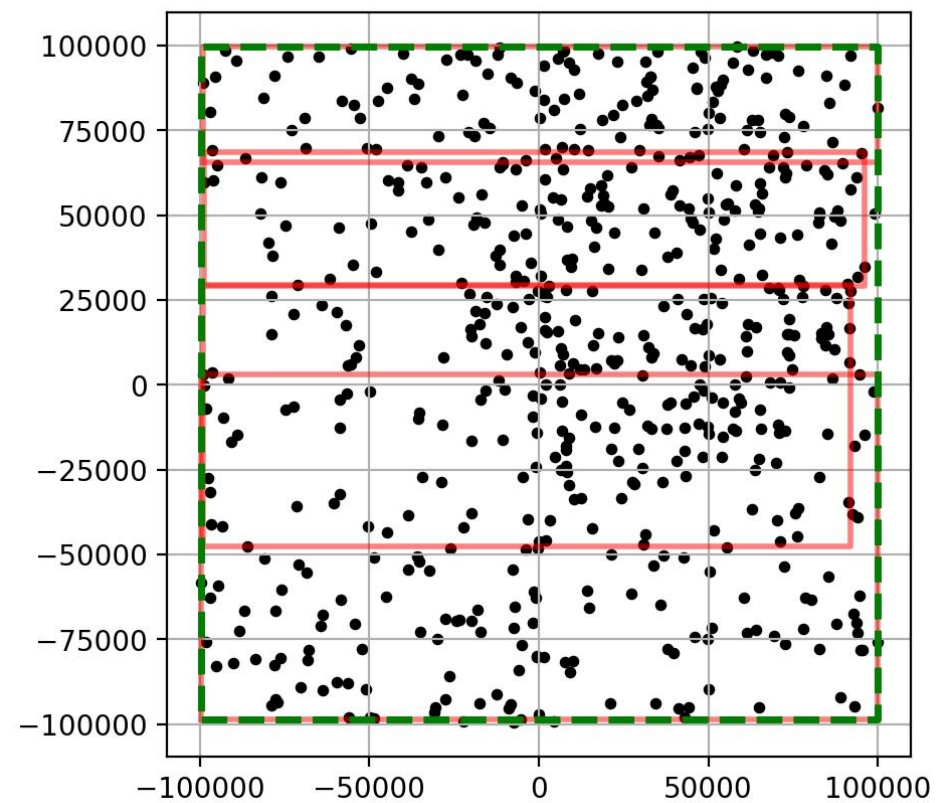
P3: Node

P4: Node

R-Tree Visualization



R-Tree Visualization



Análisis Léxico y Sintáctico (Scanner y Parser)

Utilizando técnicas de compiladores para traducir sentencias a operaciones directas en los índices.

Scanner

Convierte el texto en Tokens (palabras clave, Tipo y técnica, Identificadores, Operadores y Literales) mediante el uso de expresiones regulares.



Parser

Toma los tokens y usa la técnica de Descenso Recursivo para construir un AST (Árbol de sintaxis Abstracta) siguiendo nuestra gramática LL 1 en formato BNF. Representamos dicho AST mediante un diccionario de python.

Ejemplos:

- ('TECH', r'HASH|BTREE|RTREE|SEQUENTIAL\b')
- ('ID', r'[a-zA-Z_][a-zA-Z0-9_]*')

SQL Engine

Utilizando técnicas de compiladores para traducir sentencias a operaciones directas en los índices.

Usa el AST para verificar que la operación tenga sentido mediante:

- Type Checking
- Información de tabla guardada en el catalog
- Soporte de consultas

Gramatica BNF

```
Program → Statement [ ; ] Program | ε
Statement → CreateStmt | SelectStmt
           | InsertStmt | DeleteStmt
CreateStmt → CREATE TABLE id ( ColList )
           [ FROM FILE string ]
ColList → Column | Column , ColList
Column → id ColType [ PRIMARY KEY ]
        [ INDEX Tech ]
ColType → type | VARCHAR ( num )
        | POINT [ MAPPED BY ( id , id ) ]
Tech → HASH | BTREE | RTREE | SEQUENTIAL
SelectStmt → SELECT Selector FROM id
           WHERE id Condition
Selector → * | IdList

IdList → id | id , IdList
Condition → = Literal
           | BETWEEN Literal AND Literal
           | IN ( POINT ( num , num ) , RADIUS num )
           | IN ( POINT ( num , num ) , K num )
InsertStmt → INSERT INTO id VALUES ( ValList )
ValList → InsertVal | InsertVal , ValList
InsertVal → Literal | POINT ( num , num )
DeleteStmt → DELETE FROM id WHERE id = Literal

type → INT | DOUBLE
Literal → num | string
id → [a-zA-Z_][a-zA-Z0-9_]*
num → -?[0-9]+ ( \.[0-9]+ )?
string → " [ ^" ]* "
```

SQL Engine (Manejo de tipo espacial Point)

Utilizando técnicas de compiladores para traducir sentencias a operaciones directas en los índices.

- Al detectar una columna de tipo POINT se guarda físicamente como si fueran dos columnas DOUBLE bajo los sufijos _x y _y.
- Mapped by permite al usuario escoger el nombre de las dos columnas que componen físicamente a la columna tipo POINT, esto facilita la carga de datos de csv.

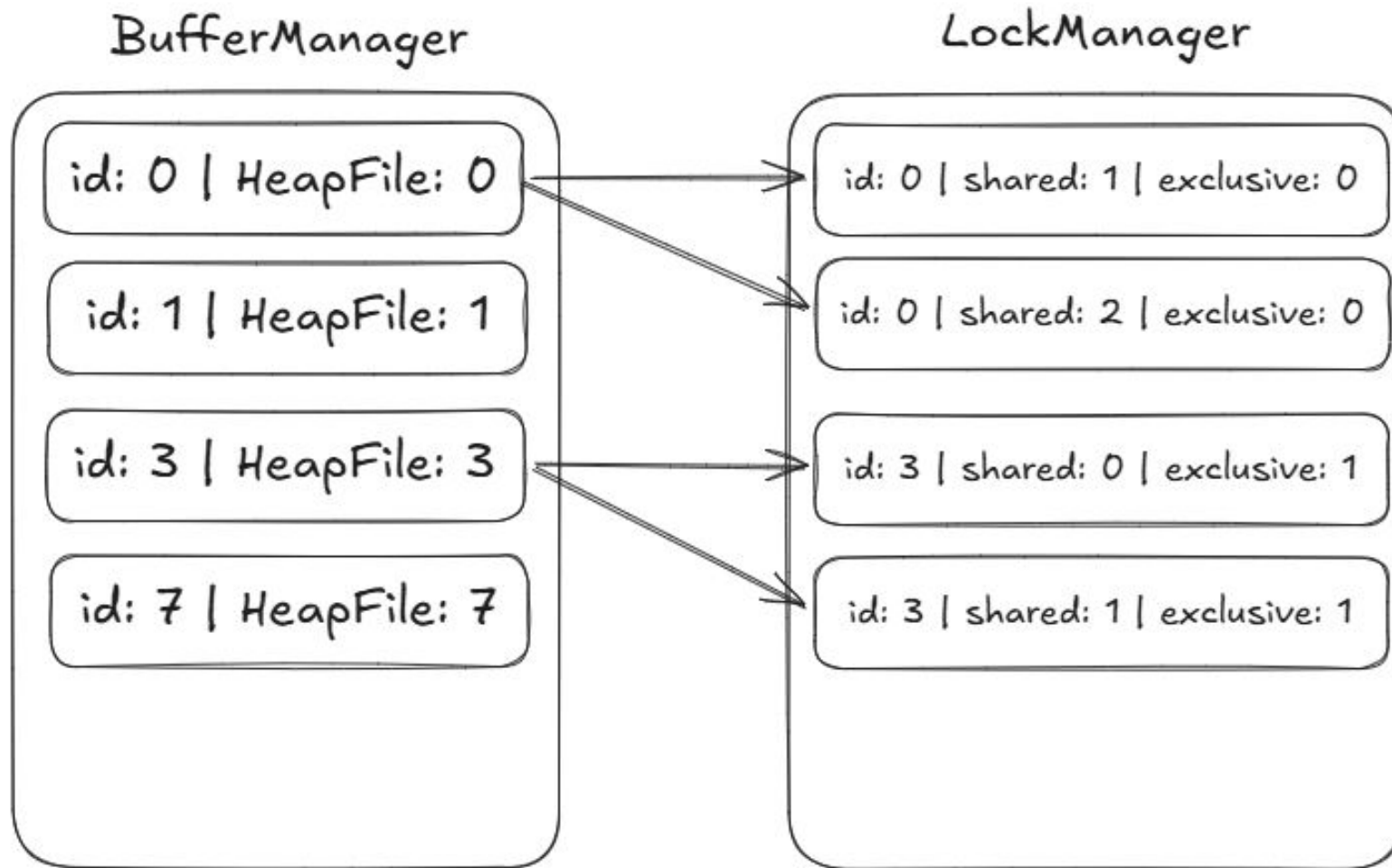
```
Iniciando DBMS SQL...
Catalog cargado exitosamente!
U-SQL> CREATE TABLE lugares (codigo VARCHAR(10) PRIMARY KEY INDEX SEQUENTIAL
, nombre VARCHAR(30), categoria VARCHAR(20) INDEX HASH, capacidad INT INDEX
BTREE, ubicacion POINT MAPPED BY (lat, lon) INDEX RTREE);
```

```
U-SQL> SELECT * FROM lugares WHERE ubicacion IN (POINT(-12.14, -77.01), K 2);

[H00K] Ejecutando Spatial Index Scan usando R-Tree para la columna 'ubicacion'.

[EXECUTE] Spatial Scan: Buscando 2 vecinos cercanos a POINT(-12.14, -77.01)...
[RTREE KNN] rows=2 | I/O reads=1 writes=0 total=1 | 0.18 ms
-> Registro: ('L-01', 'Museo MAC', 'Museo', 500, 'POINT(-12.14, -77.01)')
-> Registro: ('L-06', 'Campus UTEC', 'Universidad', 3000, 'POINT(-12.14, -77
.02)')
```

Concurrencia



Concurrencia

ACTIVITY

Results

Logs

Results

time_ms	user_id	action	detail	shared_count	exclusive_count
33.875	User 3	released	released exclusive page:3	0	0
33.978	User 3	acquired	acquired exclusive page:0	0	1
34.006	User 3	wait	wait shared page:1	0	1
40.028	User 3	acquired	acquired shared page:1	1	0
40.060	User 3	read_page	lock_test_id.idx:1		
40.106	User 3	acquired	acquired shared page:2	1	0
40.125	User 3	read_page	lock_test_id.idx:2		
40.142	User 3	released	released shared page:2	0	0
40.168	User 3	acquired	acquired shared page:2	1	0
40.185	User 3	read_page	lock_test_id.idx:2		
40.200	User 3	released	released shared page:2	0	0
40.228	User 3	acquired	acquired exclusive page:2	0	1
41.904	User 2	released	released exclusive page:1	0	0
41.940	User 2	wait	wait exclusive page:0	0	1
43.898	User 1	released	released exclusive page:0	0	0

time_ms	user_id	action	detail	shared_count	exclusive_count
82.742	User 1	acquired	acquired exclusive page:0	0	1
82.770	User 1	acquired	acquired shared page:0	1	0
82.794	User 1	read_page	lock_test_id.idx:0		
82.811	User 1	released	released shared page:0	0	0
82.840	User 1	acquired	acquired exclusive page:0	0	1
88.728	User 1	released	released exclusive page:0	0	0
91.599	User 2	released	released exclusive page:0	0	0
91.642	User 2	acquired	acquired exclusive page:0	0	1
91.665	User 2	acquired	acquired shared page:0	1	0
91.689	User 2	read_page	lock_test_id.idx:0		
91.708	User 2	released	released shared page:0	0	0
91.746	User 2	acquired	acquired exclusive page:0	0	1
99.634	User 3	wait	wait exclusive page:0	0	1
100.031	User 2	released	released exclusive page:0	0	0
100.052	User 2	released	released exclusive page:0	0	0
5053.717	User 3	wait	wait exclusive page:0	0	1
5053.752	User 3	dead lock	deadlock exclusive page:0	0	1

Muestra la línea de tiempo de concurrencia y locks.

Reiniciar

Demo en vivo

BASE DE DATOS 2

Mini DBMS / Frontend

Listo para ejecutar.

QUERY EDITOR

SQL Query

User 1 X User 2 X +

```
1 -- 3.1 Prueba de Llave Primaria (Sequential)
2 SELECT * FROM cities WHERE id = 3;
3 SELECT * FROM cities WHERE id BETWEEN 2 AND 5;
4
5 -- 3.2 Prueba de Extendible Hashing (Exacto)
6 SELECT * FROM cities WHERE state_name = "La Massana";
7
8 -- 3.3 Prueba de B+ Tree (Rangos y exacto)
9 SELECT * FROM cities WHERE state_id = 489;
10 SELECT * FROM cities WHERE state_id BETWEEN 489 AND 493;
11
12 -- 3.4 Prueba de R-Tree (Espacial)
13 SELECT * FROM cities WHERE location IN (POINT(-27.31676, -58.57197),
14   RADIUS 0.5);
15 SELECT * FROM cities WHERE location IN (POINT(-27.31676, -58.57197), K
16   2);
17
18 -- 3.5 Prueba de Full Table Scan (Columna sin índice)
19 SELECT * FROM cities WHERE name = "Riachuelo";
```

Tip: Usa multiples pestañas para organizar tus consultas.

Ejecutar todos Ejecutar

CATALOG

Datasets 1 csv

datasets/

└ cities_1.csv

Sube archivos .csv para trabajar.

Subir CSV

ACTIVITY

Results Logs Results

id	name	state_id	state_code	state_name	coun
1382	Riachuelo	3638	W	Corrientes	11

Muestra los resultados tabulares de la consulta.

Reiniciar

Gracias